

Lab 14: Linux Essentials

Brittany Rose

CITC 1332 Linux Operating System

Instructor: Dr. Noman Saied

04/09/2026

Table of Contents

14.1.....	3
14.2.....	4
14.2.1.....	5
14.2.2.....	6
14.2.3.....	7
14.2.4.....	8
14.2.4.....	9
14.2.5.....	10
14.2.6.....	11
14.2.7.....	12
14.2.8.....	13
14.2.9.....	14
14.2.10.....	15
14.2.11.....	16
14.2.12.....	17

14.1 Introduction

This is **Lab 14: Network Configuration**. By performing this lab, students will learn about the configuration of their computers on the network.

In this lab, you will perform the following tasks:

- Examine network configuration information

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 6.12.73+deb13-amd64 x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
This lab has two user accounts (username :: password )
```

```
root      :: netlab123
sysadmin  :: netlab123
```

```
Press the [Enter] key to begin...
```

```
sysadmin@localhost:~$
```

14.2 Exploring the Network

In this task, you will execute several commands and examine several files to display your network configuration.

```
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 6.12.73+deb13-amd64 x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
This lab has two user accounts (username :: password )
```

```
root      :: netlab123
sysadmin  :: netlab123
```

```
Press the [Enter] key to begin...
```

```
sysadmin@localhost:~$
```

14.2.1 Step 1

In order to determine your Internet Protocol (IP) address, execute the `ifconfig` command:

```
sysadmin@localhost:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.2 netmask 255.255.255.0 broadcast 192.168.1.255
    ether 02:42:c0:a8:01:02 txqueuelen 1000 (Ethernet)
    RX packets 40 bytes 3100 (3.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 29 bytes 2030 (2.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2 bytes 78 (78.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2 bytes 78 (78.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sysadmin@localhost:~$
```

The output shows two main blocks of information. The first block, indented by `eth0`, reflects information about your first Ethernet network card. The second block, indented by `lo`, reflects information about the *loopback* or internal network interface.

The second line in each block contains the pertinent information for version 4 of the Internet Protocol (called IPv4) while the third line has the information for version 6 of the Internet Protocol (IPv6). IPv4 is an older method of identifying machines with a series of numbers. It is still widely used today despite the fact that the improved IPv6 method has been available for many years.

The IPv4 addresses are displayed as four decimal numbers ranging from 0 to 255 separated by periods.

The IPv6 addresses are *128-bit* numbers which are displayed as hexadecimal digits ranging from 0 to f. The hexadecimal digits are generally organized into groups of four digits separated by colons. If a number of consecutive hexadecimal digits have the value of zero, then they are replaced with two colons.

14.2.2 Step 2

Having an IP address will allow your system to communicate with other systems on the same network. With *routing* devices, you are able to communicate with systems on other networks. To view the table of routing information, use the `route` command:

```
route
```

```
sysadmin@localhost:~$ route
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
default          192.168.1.1    0.0.0.0         UG    0      0      0 eth0
192.168.1.0      0.0.0.0        255.255.255.0   U      0      0      0 eth0
sysadmin@localhost:~$
```

When connecting to other computers, either an IP address or a hostname may be used. Hostnames can be used if they are entered into the `/etc/hosts` file along with their associated IP address or if a Domain Name Server (DNS) provides IP address to host name translation.

A couple of names that are commonly in the `/etc/hosts` file are `localhost`, and `localhost.localdomain`, both of which are used to refer to the current machine.

14.2.3 Step 3

Verify that the IP address 127.0.0.1 has an entry in the `/etc/hosts` file:

```
grep 127.0.0.1 /etc/hosts
```

The output should appear as follows, defining the `localhost` names:

```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2 bytes 78 (78.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2 bytes 78 (78.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sysadmin@localhost:~$ route
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
default          192.168.1.1    0.0.0.0         UG    0      0      0 eth0
192.168.1.0      0.0.0.0        255.255.255.0   U      0      0      0 eth0
sysadmin@localhost:~$ grep 127.0.0.1 /etc/hosts
127.0.0.1        localhost
sysadmin@localhost:~$
```

The `ping` command may be used to tell if a system is presently connected to a network.

Sometimes, a system may be configured to not respond to `ping` requests. Therefore, the lack of a response to a `ping` command does not mean a system is not connected to a network. A quick response to a `ping` command does indicate, however, that a system is connected to a network.

14.2.4 Step 4

Test to see if the `localhost` machine will respond to four `ping` requests:

```
ping -c4 localhost
```

```
sysadmin@localhost:~$ grep 127.0.0.1 /etc/hosts
127.0.0.1    localhost
sysadmin@localhost:~$ ping -c4 localhost
PING localhost (localhost (:::1)) 56 data bytes
64 bytes from localhost (:::1): icmp_seq=1 ttl=64 time=0.035 ms
64 bytes from localhost (:::1): icmp_seq=2 ttl=64 time=0.026 ms
64 bytes from localhost (:::1): icmp_seq=3 ttl=64 time=0.028 ms
64 bytes from localhost (:::1): icmp_seq=4 ttl=64 time=0.027 ms

--- localhost ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3060ms
rtt min/avg/max/mdev = 0.026/0.029/0.035/0.003 ms
sysadmin@localhost:~$
```

Unlike the `ping` command that is available in the Microsoft Windows operating system, the Linux `ping` command does not stop making *requests* by default. If you forget to specify the `-c` option, then you will have to manually stop the command by holding the **Control** key and pressing **C** (**CTRL+C**).

Hostnames can also be used if they are registered with a Domain Name System (DNS) server. If your system is connected to a network with DNS servers, then the `nameserver` entry in the `/etc/resolv.conf` file configures your system to use these servers to resolve hostnames into IP addresses.

14.2.5 Step 5

View the `/etc/resolv.conf` file to see if any `nameserver` entries exists:

```
cat /etc/resolv.conf
```

The output should show one `nameserver` entry:

```
sysadmin@localhost:~$ cat /etc/resolv.conf
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 127.0.0.11
options ndots:0

# Based on host file: '/etc/resolv.conf' (internal resolver)
# ExtServers: [127.0.0.1]
# Overrides: [nameservers]
# Option ndots from: internal
sysadmin@localhost:~$
```

14.2.6 Step 6

Use the `dig` command to resolve the `localhost.localdomain` name to an IP address:

```
dig localhost.localdomain
```

```
; <<>> DiG 9.11.3-1ubuntu1.13-Ubuntu <<>> localhost.localdomain
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 45661
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 4096
; COOKIE: 20ae0d586504a071511d8c9069d7ba92ebe6ec7bbc890a60 (good)
;; QUESTION SECTION:
;localhost.localdomain.      IN      A

;; ANSWER SECTION:
localhost.localdomain.  86400   IN      A      127.0.0.1

;; AUTHORITY SECTION:
localhost.              86400   IN      NS      localhost.localdomain.

;; Query time: 1 msec
;; SERVER: 127.0.0.11#53(127.0.0.11)
;; WHEN: Thu Apr 09 14:41:22 UTC 2026
;; MSG SIZE rcvd: 108

sysadmin@localhost:~$
```

Notice the output shows that the first `nameserver` that was listed in the `/etc/resolv.conf` file is the one that responded with the answer in the output (`;; SERVER: 127.0.0.11#53(127.0.0.11)`).

Note

If the above command does not execute successfully, execute the following command before re-attempting the `dig` command above. When prompted for a password, type `netlab123`:

```
sudo /etc/init.d/bind9 restart
```

14.2.7 Step 7

You can use the `dig` command to resolve other *fully-qualified domain names*. Use the `dig` command to resolve the `cserver.example.com` hostname to an IP address:

```
dig cserver.example.com
```

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 46221
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: b8c88d4fa09117188678eb7869d7baee64d806d311d53ff9 (good)
;; QUESTION SECTION:
;cserver.example.com.          IN      A

;; ANSWER SECTION:
cserver.example.com.          86400   IN      A      192.168.1.2

;; AUTHORITY SECTION:
example.com.                   86400   IN      NS      example.com.

;; ADDITIONAL SECTION:
example.com.                   86400   IN      A      192.168.1.2

;; Query time: 1 msec
;; SERVER: 127.0.0.11#53(127.0.0.11)
;; WHEN: Thu Apr 09 14:42:54 UTC 2026
;; MSG SIZE rcvd: 122

sysadmin@localhost:~$
```

A fully-qualified domain name (FQDN) includes not just the hostname, but also the domain that the hostname is "in". For the FQDN `cserver.example.com`, `cserver` is the hostname and `example.com` is the domain.

14.2.8 Step 8

Use the `dig` command to resolve the IP address `192.168.1.2` to a hostname:

```
dig -x 192.168.1.2
```

```
sysadmin@localhost:~$ dig -x 192.168.1.2

; <<>> DiG 9.11.3-1ubuntu1.13-Ubuntu <<>> -x 192.168.1.2
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 532
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;2.1.168.192.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
2.1.168.192.in-addr.arpa. 600     IN      PTR      2111f868-3d52-42e3-a815-9333e0ae
ad1c.1.2111f868-3d52-42e3-a815-9333e0aead1c.LAN.

;; Query time: 0 msec
;; SERVER: 127.0.0.11#53(127.0.0.11)
;; WHEN: Thu Apr 09 14:54:20 UTC 2026
;; MSG SIZE rcvd: 159
```

The hostname can be found in the `ANSWER SECTION` of the output of the `dig` command. The hostname in the example above looks like the following:

```
;; ANSWER SECTION:
2.1.168.192.in-addr.arpa. 600     IN      PTR      84a6444e-f044-4754-98e6-64cc9
f7dadea.1.84a6444e-f044-4754-98e6-64cc9f7dadea.LAN.
```

Although it may seem like a long hostname, this is detectable as a hostname in our virtual environment. An example of a more simple hostname may look like the following:

```
;; ANSWER SECTION:
192.168.1.2. 8640     IN      PTR      example.com
```

14.2.9 Step 9

The `netstat` command performs a large variety of tasks related to networking. To get an idea of some of its capabilities, execute the command with the `--help` option:

```
netstat --help
```

```
-n, --numeric           don't resolve names
--numeric-hosts         don't resolve host names
--numeric-ports         don't resolve port names
--numeric-users         don't resolve user names
-N, --symbolic          resolve hardware names
-e, --extend            display other/more information
-p, --programs          display PID/Program name for sockets
-o, --timers            display timers
-c, --continuous       continuous listing

-l, --listening         display listening server sockets
-a, --all               display all sockets (default: connected)
-F, --fib              display Forwarding Information Base (default)
-C, --cache            display routing cache instead of FIB
-Z, --context          display SELinux security context for sockets

<Socket>={-t|--tcp} {-u|--udp} {-U|--udplite} {-S|--sctp} {-w|--raw}
           {-x|--unix} --ax25 --ipx --netrom
<AF>=Use '-6|-4' or '-A <af>' or '--<af>'; default: inet
List of possible address families (which support routing):
  inet (DARPA Internet) inet6 (IPv6) ax25 (AMPR AX.25)
  netrom (AMPR NET/ROM) ipx (Novell IPX) ddp (Appletalk DDP)
  x25 (CCITT X.25)
sysadmin@localhost:~$
```

One of the common uses of `netstat` is to determine which services are *listening* to or waiting for an incoming connection. For example, a service that is used to allow users to perform remote or network logins is called Secure Shell or SSH. SSH normally will *listen* to TCP port 22.

Well-known ports are the port numbers in the range of 0-1023, typically used by system processes to provide network services. A list of service names and associated port numbers can be found in the `/etc/services` file.

14.2.10 Step 10

Use the `netstat` command to see if the TCP port 22, for `ssh`, has a process listening:

```
netstat -tl
netstat -tln
```

```
sysadmin@localhost:~$ netstat -tl
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 localhost:953           0.0.0.0:*               LISTEN
tcp        0      0 localhost:domain        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:40997        0.0.0.0:*               LISTEN
tcp        0      0 localhost:domain        0.0.0.0:*               LISTEN
tcp6       0      0 localhost:953           [::]:*                  LISTEN
tcp6       0      0 [::]:domain             [::]:*                  LISTEN
tcp6       0      0 [::]:ssh                 [::]:*                  LISTEN
sysadmin@localhost:~$ netstat -tln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.1:953           0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:40997        0.0.0.0:*               LISTEN
tcp        0      0 192.168.1.2:53          0.0.0.0:*               LISTEN
tcp6       0      0 :::1:953                 :::*                     LISTEN
tcp6       0      0 :::53                    :::*                     LISTEN
tcp6       0      0 :::22                    :::*                     LISTEN
sysadmin@localhost:~$
```

14.2.11 Step 11

The `-t` option to the `netstat` command limits the listing to TCP ports; the `-l` option limits the output to ports with listening services; the `-n` shows the network addresses numerically:

```
sysadmin@localhost:~$ netstat -tln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 127.0.0.1:953           0.0.0.0:*               LISTEN
tcp      0      0 127.0.0.1:53            0.0.0.0:*               LISTEN
tcp      0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp      0      0 127.0.0.11:40997        0.0.0.0:*               LISTEN
tcp      0      0 192.168.1.2:53          0.0.0.0:*               LISTEN
tcp6     0      0 :::1:953                 :::*                     LISTEN
tcp6     0      0 :::53                    :::*                     LISTEN
tcp6     0      0 :::22                    :::*                     LISTEN
sysadmin@localhost:~$
```

14.2.12 Step 12

A more modern approach is to use the `ss` command to view which connections are currently established between the local machine and remote machines, statistics about those connections, etc.

Similar to the `netstat` command, you can get a great deal of useful information from the `ss` command by itself, as shown in the example below. We will use a simple script to generate some traffic first:

```
start_webserver
```

```
sysadmin@localhost:~$ start_webserver
Connecting ...
Connecting ...
Connecting ...
Connecting ...
Connecting ...
```

Then we will use the `ss` command to display network statistics:

```
ss
```

```
sysadmin@localhost:~$ ss
NetidState      Recv-Q Send-Q   Local Address:Port      Peer Address:Port
u_dgrESTAB      0      0           /dev/log 42260544          * 0
u_dgrESTAB      0      0              * 42260547        * 42260544
u_dgrESTAB      0      0              * 42257624        * 42260544
u_dgrESTAB      0      0              * 42221502        * 42260544
tcp  FIN-WAIT-2  0      0      127.0.0.1:8000      127.0.0.1:56104
tcp  FIN-WAIT-2  0      0      127.0.0.1:8000      127.0.0.1:56106
tcp  CLOSE-WAIT  115    0      127.0.0.1:56104     127.0.0.1:8000
tcp  CLOSE-WAIT  115    0      127.0.0.1:56106     127.0.0.1:8000
tcp  CLOSE-WAIT  115    0      127.0.0.1:56090     127.0.0.1:8000
tcp  FIN-WAIT-2  0      0      127.0.0.1:8000      127.0.0.1:56090
sysadmin@localhost:~$
```

The script started a webserver and created the traffic which is displayed by the `ss` command. This is one way the `ss` command can be used for troubleshooting.